EXTENDED ABSTRACT
ACTIVITY TRACKING WITH THE REACHY MINI ROBOT

Artificial Intelligence
Lectoraat Data Intelligence
Zuyd Hogeschool

Caio de Alcantara Santos (2604500)
Cas Limpens (2400410)
Donné Deguelle (2301667)
Gustavo Gonçalves da Costa (2604824)
Kanak Manjari (Supervisor)
Maarten Vaessen (Client)

2026/2027, Q1
9-11-2026

Abstract

Children with juvenile dermatomyositis perform Childhood Myositis Assessment Scale (CMAS) exercises under hospital supervision. Which is burdensome for families. This feasibility study investigated whether the Reachy Mini robot can serve as an engaging platform for home activity monitoring under its computational constraints. Using an iterative Design Science Research inspired approach. A prototype was developed with a shared software core running identical recognition code in simulation and on the robot. The pipeline combines MoveNet Lightning pose estimation, torso-normalised temporal features over sliding windows, and NumPy-exported random forests, with real-time expressive feedback through antennas and person tracking. GPU acceleration was also investigated using LiteRT and WebGPU, but both implementations were blocked by an AES compatibility issue in the GPU runtime. Person detection was improved from 52 procent to 72 procent of frames by cropping around the previous pose. The prototype recognises four CMAS items without scoring them. The conclusion is that CPU only activity monitoring with expressive feedback is technically feasible, but clinical validation and user studies remain necessary before actual home use.

1 Introduction

1.1 Background

The Research Centre for Data Intelligence conducts applied research into, among other themes, smart assistants and human-computer interaction. This project was carried out in the context of the centre's "Measuring at Home with AI" research direction. The long-term motivation for that research is to make exercise monitoring more accessible to children with juvenile dermatomyositis (JDM). These children currently perform exercises related to the Childhood Myositis Assessment Scale (CMAS) under the supervision of specialists at the Wilhelmina Children's Hospital (WKZ). Repeated travel to the hospital is burdensome for children and their families, which creates an opportunity for technologies that can support monitoring at home. Automated home monitoring must do more than record a video. It must detect a person, estimate body movement from visual input, distinguish between activities, and communicate its interpretation clearly to the user. It must also operate under practical conditions, such as changing distances, incomplete views of the body, and limited computing resources. The stakeholders for this feasibility study are the Research Centre for Data Intelligence, the project client and researcher, and the children and families who represent the future users of a home-monitoring system. The project requirements were consolidated in a software requirements specification covering perception, robot interaction, activity recognition, feedback, session management, privacy, accuracy, reliability, usability, and safety. The current project addresses the technical feasibility of such a system; it does not develop or validate a medical assessment instrument.

1.2 Goal

The goal of this project is to investigate whether the Reachy Mini can perform person detection and activity monitoring, including expressive feedback, under the computational constraints of the wireless robot. The resulting artefact is a working prototype that runs in simulation and on the physical robot. It contains a shared camera-to-pose-to-classifier pipeline, temporal feature extraction, activity-recognition models, and robot applications that translate recognized actions into antenna responses. Candidate vision models were benchmarked under the robot's CPU constraints, leading to the selection of MoveNet int8 for on-robot pose estimation. Model-training and export tools were also developed so that the robot can perform inference with NumPy-only model files rather than requiring the full training environment. Recognition models were evaluated on public skeleton datasets with subject or footage-group separation, as well as on robot recordings and an external dataset. Users can also record personalized movements and create their own datasets for future recognition models. The prototype recognizes six classes: none, wave, pushup, squat, clapping, and jumping jacks. Evaluation focuses on technical feasibility using simulation, public skeleton datasets, recorded robot footage, an external test dataset, and different camera framings. The robot's camera should ideally capture the whole person, because waist-up framing removes information that is important for some activities. The prototype is not a CMAS scorer, is not clinically validated, and should not be used as a clinical decision-support system. The central research question is:

> To what extent can the Reachy Mini detect a person, recognize physiotherapy-related exercises, and provide expressive feedback while operating within its computational constraints?

The remainder of this report first discusses the theoretical foundations and related solution space. It then describes the prototype and its evaluation method, presents the results, discusses their implications and limitations, and concludes with directions for future development.

2 Methods

This study employed a custom, iterative methodology to investigate and develop an activity-monitoring system for the Reachy Mini. The approach shares similarities with the Design Science Research (DSR) framework, which is suitable for design-oriented research involving the development and evaluation of an artefact [1]. The approach was selected because the technical feasibility of the system was initially uncertain and several technical choices required investigation before implementation.

The methodology consisted of five phases:

1. Research: identifying stakeholder needs, establishing requirements, and investigating existing techniques.
2. Design: selecting suitable models and defining the technical architecture.
3. Realisation: implementing the activity-recognition pipeline and integrating it with the robot.
4. Testing: evaluating model performance and system behaviour under different conditions.
5. Evaluation: assessing feasibility, identifying limitations, and formulating recommendations.

The phases were not strictly sequential. Research informed design decisions, while implementation and testing were conducted iteratively. Results from experiments were used to refine technical choices and improve the artefact. This approach supported early validation and incremental improvement, which suited the exploratory nature of the project [2]. A traditional waterfall approach was considered less suitable because the technical choices were not fully determined at the start and the project required opportunities to incorporate findings during implementation [3].

Figure 1 illustrates the iterative prototyping approach. Figure 2 shows the activities conducted during the research phase.

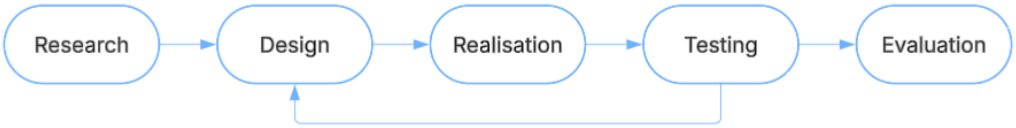


Figure 1: Iterative prototyping approach used in the project.

2.1 Requirements and software requirements specification

The research began with a stakeholder analysis to identify the parties involved and their interests and needs. Functional and non-functional requirements were derived from the project description and meetings with the client. These requirements were consolidated in a software requirements specification (SRS), which served as the reference for design and evaluation.

The functional requirements cover perception and local processing, robot behaviour and interaction, activity and exercise recognition, assessment and feedback, and session management. The non-functional requirements address privacy, recognition accuracy, reliability, usability, and safety. The complete SRS contains the detailed requirement statements and is included as an attachment to the article.

The research was organised around two complementary activities. First, existing techniques for person detection, pose estimation, and activity recognition were investigated. Second, relevant scientific literature, datasets, standards, and guidelines were studied to identify suitable approaches and relevant constraints. The findings were compared against the system requirements and the Reachy Mini's camera and computational capabilities. This process informed the selection of candidate models and the subsequent evaluation of their suitability.

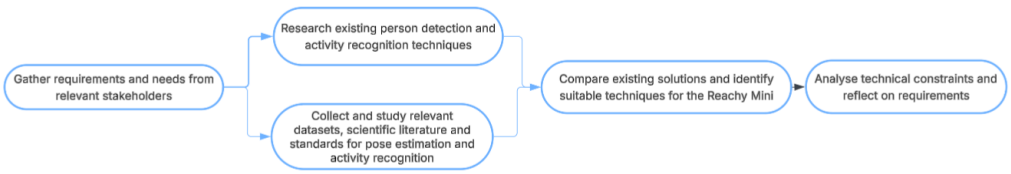


Figure 2: Research activities used to establish requirements and identify suitable techniques for the Reachy Mini.

2.2 Realisation Phase

In the realisation phase, the research was turned into a working prototype. Because the technical choices were not yet fully determined at the start, realisation did not follow the design strictly from above. But proceeded through short build-test cycles in which each design decision was validated on a functional prototype as soon as possible. A simulation first strategy was chosen because building on the physical robot was not always practical. The code was therefore organised by where it must run. A shared core that is identical in both environments, robot applications, simulation tools (a MuJoCo simulation of the Reachy Mini and a webcam-based simulation on the laptop). All dependencies point only into the core, so that one model file and one feature implementation serve both the simulation and the robot.

2.3 Testing Phase

Testing was not carried out as a single stage after the build. But was integrated into the short design cycles. Each component was checked in simulation first and then run on the physical Reachy Mini robot. In this way, problems that only appear on the real hardware were found and fixed. Pose models were benchmarked on the robot's CPU to determine which estimators are feasible, and the detection rate and keypoint confidence of the chosen pose backend were measured on robot footage and on replayed recordings. In addition to CPU benchmarking, GPU acceleration was investigated to determine whether the Reachy Mini's integrated VideoCore V3D GPU could be used for pose estimation. For the model evaluation, three data contexts were used deliberately: public datasets as the in-domain measure, recordings made with the robot itself as the realistic "robot in a room" measure, and external datasets as deliberately pessimistic checks of the domain gap. Where a class had not yet been measured on robot footage, this was recorded as not measured rather than extrapolated. Robustness to a partial view of the person was tested by removing the hip, knee and ankle keypoints from the test windows, simulating waist-up framing. The complete applications were run on the robot to verify frame rate, feedback and timing under real conditions.

2.4 Evaluation Phase

In the evaluation phase, the prototype was assessed against the requirements established in the research phase and against the central research question. The evaluation was structured as requirements check and a reflection on the research process, rather than as a clinical or user validation: the prototype is explicitly not a CMAS scorer, and engagement could not be measured because no user study was conducted. The requirements check established, per requirement, whether the prototype meets it, or does not meet it. The process reflection evaluated the chosen approach itself: which decisions contributed and which aspects limited the results.

3 Results

3.1 Description of the Artefact

The end product is a software application: a working prototype (an instantiation, in Design Science Research terms) that turns the Reachy Mini into a CPU-only activity monitor that answers what it sees with its own body. It consists of three parts: a layered software architecture that runs the same recognition code in simulation and on the robot, recognition models exported as NumPy-only files, and robot applications built on both. It addresses the requirements of the software requirements specification (SRS) attached to this article (Section 3.1).

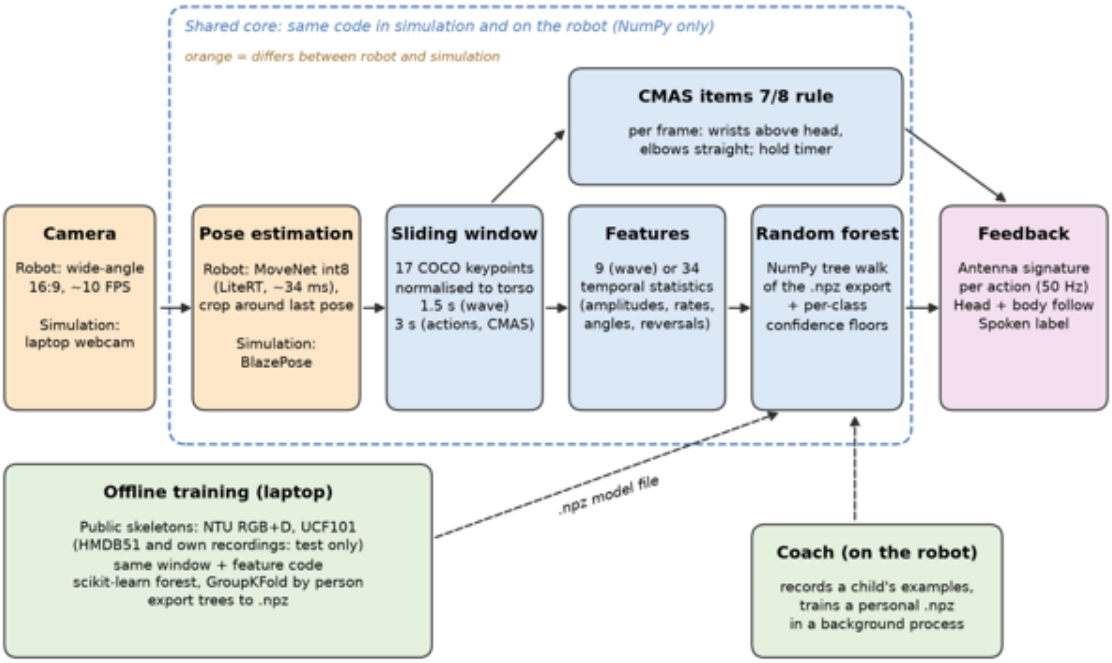


Figure 3: Pipeline Architecture.

Figure 3 represents the artefact's pipeline. Everything inside the dashed line is the same code in simulation and on the robot; only the camera and the pose estimator differ. Models are trained offline, or by the coach on the robot, and loaded as `.npz` files.

The pipeline works in such way that a camera frame goes to a pose estimator that returns 17 COCO keypoints: MoveNet Lightning int8 on the robot, BlazePose on the laptop webcam in simulation. The keypoints enter a sliding window, are normalised to the torso and summarised as temporal features. A random forest, evaluated by a NumPy-only tree walk, classifies every window, and a per-class confidence floor turns uncertain windows into *none*. Each detection drives the feedback: a distinct antenna signature per action, rendered at 50 Hz independently of the 10 FPS vision loop, head and body following that keeps the person in view, and an optional spoken label.

As for architecture and organization, the code is organised by where it must run: a shared core (pose backends, features, models, feedback), robot applications, simulation tools for MuJoCo and a webcam, and training tools, which are the only part that needs scikit-learn. Dependencies point only into the core, so one model file and one feature implementation serve both environments, and training and inference cannot drift apart.

For the final application, the robot runs action recognition (the six classes, or the CMAS items of Section 4.6), wave recognition, head and body tracking, and a coach for children. The coach guides a child through about 4.5 minutes of recording (squats, arm raises, side bends), trains a personal model on the robot's own CPU while the head keeps following, and then runs a workout with that model. A preflight tool brings the robot into a known state before every application.

Table 1 links the main decisions to the requirement or insight behind them.

3.2 Pose-Model Performance

The benchmark compared BlazePose, MoViNet-A0, TSM-MobileNetV2, ViTPose and MoveNet as candidates for the perception pipeline. MoveNet Lightning int8 reached

Table 1: Origin of the main design decisions.

Origin	Decision
Requirement: local processing on a CPU without GPU	MoveNet int8 for pose; forests exported to NumPy instead of a deep skeleton network
Requirement: privacy of child users	Only keypoints are stored; children's recordings and models stay on the robot
Grounding: windowed data leaks between splits	Every score from GroupKFold by person
Grounding: public datasets may not transfer to a room	Robot and room recordings kept as external test sets only
Design cycle: BlazePose cannot run on the robot's CPU (its binary needs AES instructions)	Pose backend made swappable behind one interface
Design cycle: a table-mounted camera often sees the person waist-up	Training windows also built with hips and legs removed
Design cycle: letterboxing the 16:9 wide-angle frame left the person a few dozen pixels tall	Pose crop centred on the previous pose; person detected in 72% of frames instead of 52%
Design cycle: scikit-learn inference cost ∼33 ms per window on the robot	Trees exported to `.npz` and walked in NumPy
Design cycle: the robot often boots asleep, motors off, camera released	Preflight with automatic fix

23.80 FPS in the inference benchmark, with a mean pose-inference time of approximately 34 ms on the Reachy Mini; in the complete application the camera, at about 10 FPS, sets the rate (Section 4.5). This was the most suitable result for the CPU-only target, and the model produces the COCO-17 keypoints the recognition code expects. Cropping each frame around the previous pose, instead of letterboxing the whole wide-angle frame, raised the share of robot frames with a detected person from 52% to 72%. On a replay of a laptop recording with the robot's aspect ratio, it raised the mean keypoint confidence from 0.48 to 0.68.

3.3 GPU Acceleration Experiments

The two GPU implementations used LiteRT and WebGPU. In MoveNet, an ARM64 LiteRT runner could be created, while in BlazePose a native C++ library as well as a GPU inspector could be built. The two implementations could be tested in runtime, but both failed at the GPU initialization step. After further investigations, it was found out that there was a compatibility problem with the prebuilt LiteRT WebGPU accelerator. It included AES instructions, which were not supported by the Reachy Mini's Cortex-A72 CPU and thus caused an illegal instruction problem. The problem appeared to be independent of the camera and model inference, meaning that it was caused by the GPU runtime and not by the models themselves. Neither of the implementations could thus reach the GPU inference step. The full code of the GPU implementations are available in the /repos section of the repository. Build results are included in Appendix B.

3.4 CMAS Items

In CMAS mode the prototype recognises that four CMAS items were performed; it does not score them. Items 12 (chair rise) and 14 (pick-up) are classified by a three-class forest (none, chair rise, pick-up) trained on NTU RGB+D alone (actions "standing up" and "pick up"). Items 7 and 8 (arms straight above the head, and for how long) use a geometric rule on single frames, because a held posture has no movement for a window to learn. Item 13 (stool step) was left out: no public dataset contains it and none was recorded.

3.5 Pose-model performance

The benchmark compared BlazePose, MoViNet-A0, TSM-MobileNetV2, ViTPose and MoveNet as candidate components for the perception pipeline. MoveNet Lightning int8 achieved an average of 23.80 FPS, with a mean pose-inference time of approximately 34 ms on the Reachy Mini. This was the most suitable result for the project's CPU-only deployment target. The model also produces the COCO-17 keypoints expected by the temporal feature and recognition code.

The comparison also exposed deployment constraints. BlazePose and ViTPose are useful comparison models, but the robot cannot use the available BlazePose MediaPipe binary because it requires AES instructions unavailable on the Reachy's Cortex-A72; ViTPose's detector-and-pose pipeline is intended for the laptop or simulation environment. MoViNet-A0 and TSM-MobileNetV2 were considered for temporal classification rather than selected as replacements for the keypoint-based pipeline. Consequently, MoveNet int8 was used for robot inference, while the shared downstream representation remained model-independent.

3.6 Wave recognition

On two recorded sessions with our own robot (416 windows), the rule-based detector reached F1 0.811 (precision 1.000, recall 0.682). The NTU-trained forest, which never saw these recordings, reached F1 0.951 and outperformed a forest trained on the sessions themselves (F1 0.884, historical only). On unseen NTU subjects, where the negatives include deliberately confusable gestures (salute, selfie and brushing hair), F1 was 0.556. On HMDB51 movie clips it was 0.065, a deliberately pessimistic external check.

3.7 Six-class action recognition

Table 2: Out-of-fold F1 on held-out subjects or footage groups. Waist-up framing is simulated by removing the hip, knee and ankle keypoints.

Class	Full body	Waist-up
none	0.902	0.855
wave	0.918	0.873
squat	0.922	0.803
clapping	0.898	0.899
jumping_jacks	0.887	0.861
pushup	0.381	0.137

These are in-domain scores. A source confound inflates wave and clapping, both taken from the studio-recorded NTU corpus. On our robot recordings wave drops to 0.581, the figure to quote for a robot in a room, and clapping has not yet been measured on robot footage. Squats are almost never false positives (precision 0.998) but miss about one repetition in seven (one in three waist-up). Push-ups (19 training groups, per-fold F1 0.06–0.71) are insufficiently measured and are not demonstrated. Without crop augmentation, waist-up F1 for wave, squat and clapping fell to 0.000.

4 Discussion

Several choices contributed positively to the results. Training on skeletons from public datasets gave the project a fast start: working recognition models existed before any data of our own had been recorded, and no video had to be processed. That head start had a limit, however. The public datasets did not cover every exercise the project needed, and they did not show a person the way the robot sees one, so we later had to record our own sessions to train and test the models that mattered most. A second positive choice was to run the same code in simulation and on the robot. Features, models and feedback were developed and checked on a laptop, then deployed to the robot unchanged, which kept the two environments from drifting apart. Finally, short design cycles on the physical robot exposed problems that no laptop test would have shown. They revealed that BlazePose cannot run on the robot's processor, and that letterboxing the wide-angle image left the person too small to detect reliably. Each problem was found and fixed within days, and the crop around the previous pose raised the share of frames with a detected person from 52 percent to 72 percent.

Other aspects limited the results. The robot's camera does not match the image quality of a laptop or phone camera. Its wide-angle view leaves a person only a small part of the image, so keypoints are less certain than in simulation. The CPU, without a GPU, also restricted the choice to a lightweight pose model. LiteRT and WebGPU setups have been tested for GPU acceleration, but both were built to ARM64 level and had problems initializing the GPU because of unsupported AES instructions in the accelerator. A further limit was the difference between the training videos and real use. The public datasets were recorded in studios or taken from films, with other backgrounds, camera angles and sometimes several people in view. The robot, in contrast, sees one person in a home room from table height. This difference lowered recognition quality outside the datasets: waves, for example, scored F1 0.918 on dataset subjects but 0.581 on our robot recordings. The robot was also not always available to the team. The simulation largely mitigated this risk, because most of the development could continue without it. Lastly, engagement could not be measured, because the prototype could not be tested with real users. Whether children find the robot an engaging exercise buddy therefore remains an open question.

5 Conclusion

The goal was to find out whether the Reachy Mini can detect a person, recognise their activity and give expressive feedback within the limits of its own CPU. GPU acceleration was also investigated through two LiteRT/WebGPU implementations using MoveNet and BlazePose. Although both implementations were successfully prepared for ARM64, neither reached GPU inference because the shared WebGPU accelerator contained AES instructions unsupported by the Reachy Mini's Cortex-A72 processor. This established a concrete compatibility limitation and provides a starting point for future work. Rebuilding the accelerator for the target processor is the most direct next step. The complete pipeline runs on the robot's Raspberry Pi CM4 without a GPU or a laptop. Pose estimation takes about 34 ms per frame, features and forest a few milliseconds, and the camera's 10 FPS, not the computation, sets the pace. The same code and models run in simulation, so they are developed on a laptop and deployed unchanged. Recognised actions are answered in real time with antenna signatures and head and body following.

The answer to "how well" is more limited. In live use, wave, clapping and jumping jacks were reliable enough to demonstrate, and squats were recognised with about one repetition in seven missed. Push-ups were not. Scores on public datasets overstate what happens in a room: the six-class model recognises waves with $F_1 = 0.918$ on dataset subjects but $F_1 = 0.581$ on our robot recordings. For the CMAS items, pick-up held up in a room ($F_1 = 0.933$). Chair rise and arms above the head were recognised live on the robot, but they have not yet been measured on a correctly performed room recording. The robot's position also matters: it must see the whole person, and a waist-up view costs recognition.

The prototype, its models, the training and export tools, and the coach were realised. A CMAS score, the stool-step item and clinical validation were not; they were outside the scope. The "engaging" part of the research question was not evaluated, since no user study was done. The prototype is therefore a research platform: it shows that monitoring at home with this robot is technically feasible, but it cannot yet be used to monitor children with JDM.

That said, there are four recommended next steps:

1. Record labelled sessions in real rooms through the robot's own camera, following the CMAS instructions, and use them as the reference for room performance.
2. Collect data for the classes that lack it: push-ups, clapping on robot footage, and the stool step.
3. Run a small user study with children and families to measure engagement, which the research question assumes, but this project did not test.
4. If CMAS scoring becomes the goal, define the scoring criteria with the WKZ specialists and validate the system against their scores.

References

[1] A. R. Hevner, "A three cycle view of design science research," *Scandinavian Journal of Information Systems*, vol. 19, no. 2, pp. 87–92, 2007. [Online]. Available: https://aisel.aisnet.org/cgi/viewcontent.cgi?article=1017&context=sjis

[2] GeeksforGeeks. "What is prototyping? definition, types, qualities and more," Accessed: Oct. 26, 2026. [Online]. Available: https://www.geeksforgeeks.org/software-engineering/what-is-a-prototype-and-how-to-create-it/

[3] Agile Scrum Group. "Agile vs waterval: Wanneer gebruik je welke methode?" Accessed: Oct. 26, 2026. [Online]. Available: https://agilescrumgroup.nl/agile-vs-waterval/

Appendices

Appendix A: System Requirements Specification

M-ADSAI-ZM002-26

System Requirements Specification

Caio De Alcantara Santos (2604500), Cas Limpens (2400410), Donné Deguelle (2301667) and Gustavo Gonçalves Da Costa (2604824).

Contents

Introduction 3
Purpose 3
Scope 3
Intended Users 3
System Overview 4
Product Description 4
Users and Stakeholders 4
Assumptions and Dependencies 4
Functional Requirements 5
Perception and local processing 5
Robot behaviour and interaction 5
Activity and exercise recognition 6
Assessment and feedback 6
Session management 7
Non-Functional Requirements 8
Bibliography 9

Introduction

Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements for a system that uses the Reachy Mini robot and AI-based movement detection to monitor and evaluate human movements during physiotherapy exercises at home. The project investigates whether the Reachy Mini can be used as an interactive and motivating exercise buddy. The system should be able to detect and potentially score common human movements and provide feedback or encouragement to the user. The SRS defines the functional and non-functional requirements of the proposed system and serves as a basis for its development and evaluation.

Scope

The scope of this project is to investigate the capabilities of the Reachy Mini for person detection and activity monitoring. The system will focus on detecting common movements such as walking, running, sitting, standing, and other relevant physiotherapy movements. The project will investigate whether existing AI and pose-estimation models can identify the movement being performed, determine whether it is performed correctly, and potentially provide a score. The Reachy Mini may also provide interactive feedback through movements, sounds, or voice to increase motivation. The project does not aim to solve a specific medical case or provide a complete medical monitoring solution. Instead, the focus is on investigating whether the Reachy Mini can identify and score general human movements and function as a motivating device for home-based physiotherapy.

Intended Users

The primary intended users are patients performing physiotherapy exercises at home. The system is intended to support users who perform exercises independently by providing motivation and potentially feedback on their movements. The Reachy Mini is intended to act as an interactive exercise buddy that can react to the user's movements and provide encouragement or feedback. The system may also be relevant to physiotherapists and healthcare professionals, who could use the Reachy Mini as a replacement assistant for appointments and scoring, to support the monitoring of patients' exercises. The original project document specifically mentions children with JDM (Juvenile Dermatomyositis) performing exercises that are assessed using the CMAS (Childhood Myositis Assessment Scale), but this project broadens the current project goal toward general physiotherapy movements and investigating the Reachy Mini as a motivating device.

System Overview

Product Description

The product is a movement detection system that uses the Reachy Mini robot's capabilities and AI algorithms to detect and score JDM patients movement during physiotherapy exercises at home. The system aims to provide motivation to patients. To support exercise performance.

Users and Stakeholders

The intended users of the Reachy Mini are JDM patients. But also, healthcare professionals who assess these patients. The robot could help assess the patients, without the patients needing to travel to the hospital as often.

Our client Maarten Vaessen who has tasked us with this project, is a senior researcher in AI for healthcare at Zuyd research center data intelligence (Zuyd University of Applied Sciences, n.d.).

Assumptions and Dependencies

The Reachy Mini can be programmed with the use of the Reachy Mini python SDK. Which is supported on Windows, MacOS and Linux. The python SDK requires python version 3.10-3.12 and the latest version of git, including git LFS (Huggingface, n.d.). It is assumed that the client will provide the Reachy Mini DIY kit.

Functional Requirements

The functional requirements describe what the system has to do. Each requirement has a unique identifier, is written using the modal verb *shall*, and applies to a single user in front of a single Reachy Mini during an exercise session.

Several requirements refer to *supported activities*. This means the closed set of movements the system is able to recognise: standing, sitting, walking, running, arm raise and sit-to-stand. The set is deliberately small, because the aim of the project is to find out whether recognition of this kind is feasible on the Reachy Mini, not to cover the full range of physiotherapy exercises.

Perception and local processing

Before an activity can be recognized, the system must detect and track the user. The activity recognition approach may either use an explicit skeletal or pose representation or infer activities directly from image or video data. Both approaches are therefore considered within the scope of this research. FR-02 additionally requires that this happens locally, which is the constraint that gives the project its research character and from which the privacy property in NFR-01 follows.

Table 1 – Perception and local processing functional requirements

ID	Title	Requirement
FR-01	Person Detection	The system shall detect a person within the Reachy Mini's camera view within 5 seconds after the person enters the camera view during an exercise session.
FR-02	Local Processing	The system shall process captured camera data entirely on the Reachy Mini or the connected local computer, without requiring external processing for core activity-tracking functionality.
FR-03	Person Tracking	The system shall track the detected person continuously while the person remains visible within the camera's field of view during an exercise session.
FR-05	Pose Estimation	The system shall provide the input representation required by the selected activity recognition approach, which may be an explicit pose/skeletal representation or image/video data.
FR-18	User Detection Loss	The system shall detect that the user is no longer visible to the camera within 2 seconds after the user leaves the camera's field of view during an exercise session.

Robot behaviour and interaction

The robot moves for two different reasons. In FR-04 the head moves in order to keep the user in view, which is a perception function. In FR-13 and FR-14 the head and antennas move in order to communicate with the user, which is an interaction function. How these two influence each other is one of the questions the project sets out to answer.

Table 2 – Robot behaviour and interaction functional requirements

ID	Title	Requirement

FR-04	User Following	The system shall move the Reachy Mini's head to follow the detected user while the user remains within the camera's field of view during an exercise session.
FR-12	Audio Feedback	The system shall provide audio feedback through the Reachy Mini's speaker whenever an audio feedback response is triggered.
FR-13	Head Movement Feedback	The system shall use movements of the Reachy Mini's head to provide feedback to the user whenever a head-movement feedback response is triggered.
FR-14	Antenna Movement Feedback	The system shall use movements of the Reachy Mini's antennas to provide feedback to the user whenever an antenna-movement feedback response is triggered.

Activity and exercise recognition

Three categories are distinguished because they place different demands on the recognition approach. Static positions can be classified from a single pose, dynamic movements require a sequence of poses over time, and physiotherapy exercises additionally have a defined correct execution.

Table 3 – Activity and exercise recognition functional requirements

ID	Title	Requirement
FR-06	Static Movement Recogntion	The system shall recognise all static positions listed as supported activities during an exercise session.
FR-07	Dynamic Movement Recognition	The system shall recognise all dynamic movements listed as supported activities during an exercise session.
FR-08	Physiotherapy Exercise Recognition	The system shall recognise all physiotherapy exercises listed as supported exercises during an exercise session.

Assessment and feedback

Recognising a movement is not enough on its own, since the robot is meant to act as an exercise buddy. FR-09 and FR-10 cover the judgement about the execution, and FR-11 covers the decision to respond. The assessment is explicitly non-clinical: the movement criteria describe geometric properties such as joint angles and symmetry, and do not constitute a medical score.

Table 4 – Assesment and feedback functional requirements

ID	Title	Requirement
FR-09	Movement Assessment	The system shall assess the execution of supported physiotherapy exercises using the predefined movement criteria for each exercise.
FR-10	Repetition Detection	The system shall detect completed repetitions according to the predefined repetition criteria during an exercise session.

Session management

An exercise session is the unit of operation of the system: the user starts it, the system observes continuously during it, and the results are reported at the end.

Table 5 – Session management functional requirements

ID	Title	Requirement
FR-11	Session Management	The system shall allow the user to start and end an exercise session.
FR-12	Exercise Monitoring	The system shall monitor the user's movements continuously throughout an active exercise session.
FR-13	Exercise Result	The system shall provide the user with the detected exercise results and assessment results at the end of an exercise session.

Non-Functional Requirements

The non-functional requirements describe the qualities the system has to exhibit rather than the functions it has to perform. Each is expressed as a measurable target, so that the outcome of the investigation can be reported objectively.

Privacy is a driving constraint rather than a secondary concern, because the system observes a patient exercising at home. Accuracy determines whether the system is useful at all, since feedback based on a misrecognised movement is worse than no feedback; the threshold of 80% demonstrates technical feasibility within the scope of this project and is not a level that would be sufficient for clinical use. Reliability and usability follow from the fact that the system is operated by a patient without technical support present, and safety follows from the robot moving in the physical presence of a user who may be a child.

Table 6 – Non-Functional Requirements

ID	Title	Requirement
NFR-01	Privacy	The system shall ensure that captured video and session data remain on the Reachy Mini and/or the connected local computer for the entire duration of system operation and shall not be transmitted to external systems.
NFR-02	Recognition Accuracy	The system shall provide activity recognition for the supported activities when evaluated using the predefined test dataset. The recognition performance shall be measured and reported.
NFR-03	Reliability	The system shall complete an exercise session without requiring an application restart under normal operating conditions.
NFR-04	Usability	The system shall allow first-time users to start an exercise session without assistance from the development team.
NFR-05	Safety	The system shall not perform unexpected or uncontrolled robot movements during normal operation.

Bibliography

Huggingface. (n.d.). *Installation Guide*. Retrieved from huggingface.co: https://huggingface.co/docs/reachy_mini/SDK/installation?install-uv=Windows&install-git=Windows&install-method=%F0%9F%93%A6+Option+A%3A+PyPI

Zuyd University of Applied Sciences. (n.d.). *Data intelligence*. Retrieved from zuyd.nl: https://www.zuyd.nl/en/research/centres/data-intelligence

Appendix B: GPU Acceleration Research

M-ADSAI-ZM002-26

Reachy Mini: GPU Acceleration Research

Caio De Alcantara Santos (2604500), Cas Limpens (2400410), Donné Deguelle (2301667) and Gustavo Gonçalves Da Costa (2604824).

Contents

Introduction 3
Purpose 3
Scope 3
Research Questions 4
Methodology 5
Hardware and Technical Context 5
GPU Implementations 6
GPU Implementation 1: MoveNet with LiteRT and WebGPU 6
GPU Implementation 2: Native BlazePose with LiteRT and WebGPU 7
Shared LiteRT WebGPU Runtime Failure 8
Future Development and Recommendations 9
Conclusion 10
References 10

Introduction

Purpose

The purpose of this research is to investigate the feasibility of using the integrated GPU of the Reachy Mini to accelerate AI inference, with a particular focus on human pose estimation. This research builds upon the earlier Reachy Mini: Hardware & Technical Capability Assessment (De Alcantara Santos, Limpens, Deguelle, & Gonçalves Da Costa, 2026), which examined the robot's hardware specifications, computing capabilities, available software interfaces and suitability for running AI models locally. This research builds upon those findings by focusing specifically on GPU acceleration. Different frameworks and implementations are investigated to determine whether the Broadcom VideoCore V3D GPU can be used for machine learning inference and which technical limitations prevent successful execution.

Scope

The scope of this research is GPU acceleration of human pose estimation on the Reachy Mini. The goal of the research is not to build a complete GPU-accelerated application. It is rather an attempt to document the process of development, establish how far each implementation goes and understand what needs to be done for successful GPU inference in the future.

Research Questions

The following research questions define the focus of this research.

Main Question

“To what extent is it possible to accelerate human pose estimation on the Reachy Mini using its integrated Broadcom VideoCore V3D GPU, and what technical limitations prevent successful GPU inference?”

Subquestions

1. What are the hardware capabilities and limitations of the Reachy Mini's Raspberry Pi Compute Module 4, and which GPU acceleration frameworks are suitable for this hardware?
2. How can LiteRT and WebGPU be configured and cross-compiled for ARM64 to run MoveNet and BlazePose on the Reachy Mini?
3. What are the differences between the MoveNet LiteRT/WebGPU implementation and the native C++ BlazePose LiteRT/WebGPU implementation?
4. What caused the AES-related runtime failure in both LiteRT/WebGPU implementations, and how was the problem identified?
5. What technical improvements and development steps are required to achieve successful GPU inference on the Reachy Mini in future work?

Methodology

The research has been conducted by iterative implementation, cross-compilation and testing on the Reachy Mini. The GPU applications have been prepared initially on the Ubuntu x86_64 development machine using ARM64 cross-compilation toolchain. Then the resulting binaries and libraries have been tested on the Reachy Mini. In case of failure of GPU initialization additional diagnostics have been performed to find out the processor capabilities, binary instructions, shared library dependencies and runtime loading.

Hardware and Technical Context

The capabilities of the Reachy Mini's hardware were analyzed thoroughly in a previous work entitled Reachy Mini: Hardware & Technical Capability Assessment (De Alcantara Santos, Limpens, Deguelle, & Gonçalves Da Costa, 2026). The robot uses a Raspberry Pi Compute Module 4 with a Broadcom BCM2711 quad-core Cortex-A72 processor, 4 GB of RAM and an integrated Broadcom VideoCore VI GPU (Raspberry Pi, sd). The robot also has IMX708 camera, which serves as a source of video input needed for pose estimation. The prior work also describes other components that can be found on the robot such as sensors, cameras, software development kits, APIs, and networking capabilities.

In this work, the GPU architecture and its software compatibility are the crucial part. The VideoCore GPU can be accessed through Mesa graphics stack and V3D driver(Mesa Project, sd). As the hardware doesn't support NVIDIA CUDA, the alternatives like LiteRT and WebGPU have been analyzed. The following sections describe these technologies and the problems arising from the attempt to use them on the Reachy Mini.

GPU Implementations

GPU Implementation 1: MoveNet with LiteRT and WebGPU

The first dedicated GPU approach focused on MoveNet Lightning INT8 (STMicroelectronics, sd). It was supposed to employ the same model as the future working proof of concept for CPU but with the replacement of the XNNPACK CPU backend with GPU acceleration of the model provided by LiteRT. The latter is Google's runtime for on-device machine learning inference with the APIs for loading models and choosing hardware accelerators supported (Google AI Edge, sd). The envisioned architecture involved feeding the camera into the Python application, which was supposed to preprocess the frames into a 192 × 192 RGB tensor, pass it to the native ARM64 LiteRT runner and get the keypoints to draw a skeleton on top of the camera image.



Figure 1 Intended architecture of the MoveNet LiteRT/WebGPU implementation on the Reachy Mini

The LiteRT runtime was built cross-platform on an Ubuntu 24.04 x86_64 development machine using the aarch64-linux-gnu toolchain. In particular, an initial problem arose due to the attempt to run the ARM64 version of the FlatBuffers compiler (flatc) on the x86_64 host platform. It led to an Exec format error. The problem was solved by compiling flatc as a native x86_64 host tool and configuring the build to use that one. With this fix in place, the ARM64 runner was successfully built. It resulted in the files including the LiteRT runner, libLiteRt.so, the prebuilt libLiteRtWebGpuAccelerator.so and the MoveNet INT8 model.

The Python application was then hooked up to the runner. GPU acceleration was explicitly requested in the accelerator configuration including --accelerator=gpu, --gpu_backend=webgpu and --require_registered_accelerator=gpu options. The last one was crucial since it avoided silent falling back to the CPU.

The main problem was GPU initialization. The failure occurred during the first GPU registration check when the MoveNetLiteRT initialization routine was invoked. Specifically, the ARM64 LiteRT runner tried to load the prebuilt WebGPU accelerator library, which was compiled with the AES instructions not supported by the Reachy Mini's Cortex-A72 processor. It caused SIGILL ("illegal instruction" error), leading to failure of the initialization routine. Since the application checks that before connecting to the Reachy Mini camera, the camera processing, image preprocessing and MoveNet inference were not even attempted.



Figure 2 LiteRT WebGPU initialization failure during accelerator registration

GPU Implementation 2: Native BlazePose with LiteRT and WebGPU

The second GPU implementation used the BlazePose Full framework and followed another strategy compared to the MoveNet implementation. Instead of using an existing command-line runner, a custom native C++ inference pipeline was implemented to have greater control over the model execution. The implementation used the original MediaPipe Pose Landmarker Full model bundle, which consisted of two separate TFLite models: a pose detector and a pose landmark model (Google AI Edge, sd). The detector locates a person and defines a region of interest (ROI), and then the landmark model infers the body pose. The original pose_landmarker_full.task bundle was separated into pose_detector.tflite and pose_landmarks_detector.tflite. Both models were loaded separately using LiteRT and set up to run with the GPU backend. The intended processing pipeline was to take a frame from the Reachy Mini camera, transfer it from Python to the native C++ library through ctypes, run the detector on the GPU, calculate the ROI in C++ and then run the landmark model on the GPU. After that, the resulting landmarks should be transferred back to Python to visualize the skeleton and run benchmarking.



Figure 3 Native BlazePose inference architecture using two LiteRT GPU models and a custom C++ processing pipeline

The native implementation was split into the Python interface and the C++ processing part. The Python adapter, which was implemented in gpu_models/__init__.py, loads libblazepose_gpu.so through ctypes and invokes the exported C functions. The C++ library provides the API for creating the GPU pipeline, processing an image, getting the error information and destroying the allocated resources. This allowed to keep using the existing Python benchmark while providing the new implementation, without modifying the original CPU-based one. At the initialization stage, the C++ code creates a LiteRT environment and the GPU execution options. It explicitly enables the GPU acceleration using SetHardwareAccelerators(HwAccelerators::kGpu). Then both the detector and landmark models are loaded using CompiledModel::Create(). Their input and output tensor buffers are created once and stored in the native BlazePoseGpu object. This means that the models and their execution contexts are supposed to remain loaded between camera frames, and not to be recreated at every inference.

At the inference stage, Python converts each camera frame from BGR to RGB and sends the resulted image data to the native library. In C++, the image is resized to 224 × 224 pixels using the letterboxing technique and normalized to the range [-1, 1]. Then the detector runs through LiteRT with the GPU backend. The output includes detection boxes and confidence scores, which are then copied to the C++ memory and further processed using the anchor decoding and NMS. The top-scoring detection is used to calculate the ROI, which consists of the position, scale and rotation of it. Then the region of the image is cropped and rotated to a 256 × 256 input of the landmark model. This second model also runs through LiteRT GPU backend. The output is read back to the C++ memory, and then the coordinates of the first 33 public pose landmarks are converted back to the original image coordinates. Finally, the native library sends the coordinates of the landmarks along with the confidence values to Python, where the skeleton is

drawn on the camera image. The benchmark also collects inference latency, FPS, CPU and RAM usage.

aarch64-linux-gnu-g++ cross-compiler. CMake was configured with CMAKE_SYSTEM_PROCESSOR=aarch64 and the corresponding ARM64 compiler. The compilation depended on the existing LiteRT source and build environment located under ~/LiteRT, including the already built ARM64 LiteRT libraries. An initial error during compilation was related to the aggregate initializer used for initializing the BlazePoseGpu structure with the wrong number of fields. The structure needed an explicit constructor, which accepts a LiteRT environment and two compiled models. This was fixed by using the constructor directly and moving the model objects with std::move(). After this modification, the compilation finished successfully producing libblazepose_gpu.so and blazepose_gpu_inspector. Both binaries were later verified using the Linux file command and proved to be compiled for ARM aarch64.

After the successful cross-compilation, the next step was to test the GPU implementation directly on the Reachy Mini. For this purpose, a separate executable blazepose_gpu_inspector was built. It initializes the LiteRT environment, requests the GPU acceleration, tries to create a CompiledModel for each of the two BlazePose models and checks whether the tensor buffers use the WebGPU memory. The Python benchmark uses almost the same initialization sequence. First, it loads the native library through ctypes and calls blazepose_gpu_create(). This function creates the LiteRT environment, sets up the GPU execution and tries to initialize the detector and landmark models. Importantly, this initialization happens before the benchmark starts its camera warm-up and frame-processing loop. At the first GPU model initialization, LiteRT tries to load the pre-built libLiteRtWebGpuAccelerator.so library. The same AES-related illegal-instruction error occurred, which was observed in the MoveNet implementation. The Cortex-A72 processor of the Reachy Mini does not support these instructions of this binary, so the process terminates before the first CompiledModel can be created successfully. As a consequence, neither detector nor landmark models reach the inference stage, and no GPU tensor buffers or pose landmarks are generated. The successful ARM64 compilation thus proves that the custom application and its native binaries can be built for the Reachy Mini, but it doesn't prove that the whole GPU pipeline can run on it.



Figure 4 BlazePose LiteRT/WebGPU initialization failure caused by unsupported AES instructions on the Reachy Mini

Shared LiteRT WebGPU Runtime Failure

One of the important discoveries made in this case was that the implementation of MoveNet and BlazePose failed not due to two distinct issues related to the model. In both implementations, the pre-built LiteRT WebGPU accelerator was used - libLiteRtWebGpuAccelerator.so. This was checked by comparing the SHA-256 hashes of the accelerator files in the project, and the hashes matched, meaning that the two implementations have one and the same dependency runtime. The capabilities of the Reachy Mini processor were determined with /proc/cpuinfo command, and the output showed no support of AES and PMULL

instruction sets. Using objdump, it was seen that the library had the instructions such as aesd, aese, aesmc and aesimc, which require certain processor features absent in the target CPU architecture. Although the library is compiled for ARM64, its instruction requirements are not compatible with the specific Cortex-A72 configuration.

This was confirmed by a more direct Python ctypes.CDLL() test - the test tried loading the WebGPU library without initializing the camera, loading the model and conducting inference, but resulted in the same fatal error. Checking the WebGPU library with ldd showed that the standard dependencies of a shared library were correctly loaded, but the main libLiteRt.so library did not exhibit the problem with the AES instruction. This way, the incompatibility was isolated to the supplied WebGPU accelerator.

This is why the problem was not fixed with a change of the Python application or rebuilding the native runner, because the problem with the accelerator was due to the fact that it was supplied as a precompiled library. The LiteRT and WebGPU implementations therefore halted at the same initialization point. The next step would be to rebuild or replace the accelerator and its dependencies compiled with appropriate compiler settings for Cortex-A72. Still, even after successful library loading, WebGPU device selection, model loading and inference will have to be checked.

Future Development and Recommendations

In order to move forward with the LiteRT/WebGPU projects, we propose that the WebGPU accelerator and its dependant components be recompiled for the Cortex-A72. This way, all the necessary ARM64 components from the current implementation would be used, while only the pre-built accelerator would need to be reworked. Firstly, the version of the LiteRT source code needs to be found, then the accelerator should be recompiled under the right ARMv8 settings. The resulting binaries need to be checked for unsupported AES instructions (this can be done using objdump). It would also be helpful to check whether the accelerator works independently before trying to run a full pose estimation app using it. For future developers, it would be useful to make sure that the library loads, that the GPU accelerator is registered by LiteRT and that at least a simple model can be initialized and executed. This way, problems associated with model execution and camera initialization would not be confused with each other.

Conclusion

The greatest success was reached while working on the two LiteRT/WebGPU approaches. In particular, the MoveNet approach succeeded in building an ARM64 LiteRT runner and including it in a Python project. The second approach managed to go further in terms of native implementation and created a custom inference pipeline in C++, ARM64 shared library and GPU Inspector. At the same time, both LiteRT/WebGPU approaches stopped at the GPU initialization stage since they used the same prebuilt WebGPU accelerator which had instructions supported by the AES algorithm which is incompatible with the Cortex-A72 processor of the Reachy robot. It was proved by performing CPU feature inspection, analyzing the binary code and an additional library loading test. Thus, the primary output of the current study is a clear diagnosis of the LiteRT/WebGPU incompatibility issue and a technical base for future research. Modification of the WebGPU accelerator with appropriate compiler options is the most promising way to go forward.

References

Mesa Project. (n.d.). *V3D*. Retrieved from Mesa 3D Graphics Library: https://docs.mesa3d.org/drivers/v3d.html

De Alcantara Santos, C., Limpens, C., Deguelle, D., & Gonçalves Da Costa, G. (2026). *Reachy Mini: Hardware & Technical Capability Assessment*.

Google AI Edge. (n.d.). *Getting started with LiteRT*. Retrieved from Google for Developers: https://developers.google.com/edge/litert/overview

Google AI Edge. (n.d.). *Pose landmark detection guide*. Retrieved from Google for Developers: https://developers.google.com/edge/mediapipe/solutions/vision/pose_landmarker

Raspberry Pi. (n.d.). *Compute Module hardware*. Retrieved from Raspberry Pi Documentation: https://www.raspberrypi.com/documentation/computers/compute-module.html

STMicroelectronics. (n.d.). *MoveNet*. Retrieved from GitHub: https://github.com/STMicroelectronics/stm32ai-modelzoo/tree/main/pose_estimation/movenet